

Furthering GUI-Grounding for Agentic AI: An Evolving Training Dataset

André Shannon

advised by
Dr. Isabel Song

May 5, 2026
Department of Computer Science
San Francisco State University

Overview

- Introduction
- Related Work
- Problem Statement
- Method
- Experiment
- Conclusion and Future Work

Introduction

- **Agentic AI** - an autonomous agent that can navigate a computer/web and complete complex multi-step tasks.
- **GUI grounding** - mapping natural-language instructions to specific screen elements.

Related Works: SE-GUI

Key contributions of SE-GUI:

- Curated a 3k dataset
- Defined a more informative reward function
- Developed a **self-evolutionary** mechanism that iteratively refines their training dataset using attention maps

Achieved state of the art results for their 3B and 7B parameter models with only 3k of training data (orders of magnitude less data than others).

SE-GUI: Self-Evolution

Stage 1:

- Train model like normal on full dataset for N epochs

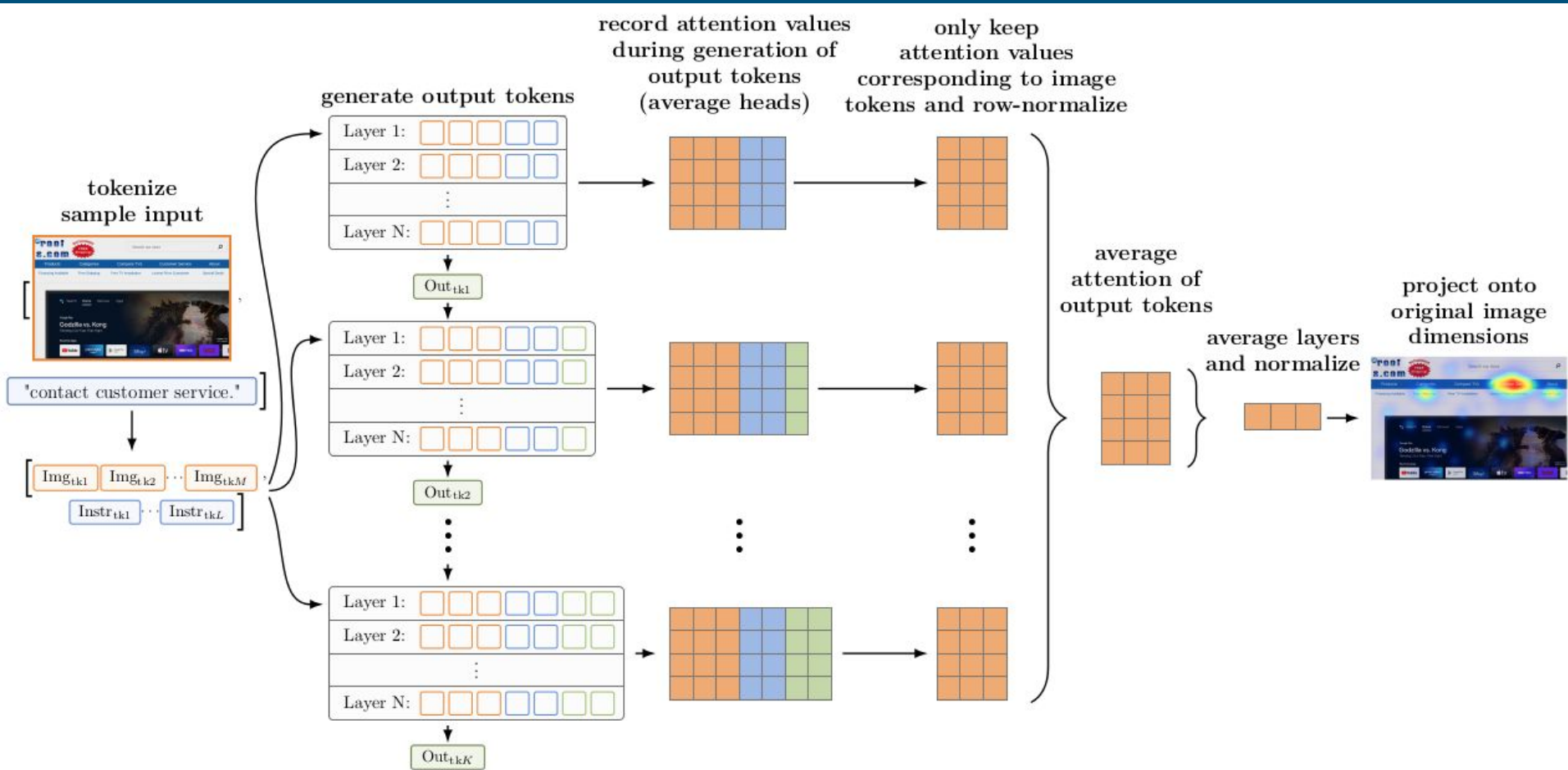
Stage 2+:

- **Filter** the original dataset using the attention map of the previous stage's trained model
- Train for another N epochs on the filtered dataset

SE-GUI: Self-Evolution: Filtering

For each sample:

- Run it through the previous stage's trained model and record transformer attention values of generated tokens to the input image
- Normalize the attention vector for each generated token and average across all generated tokens and model layers.
- Project attention values onto original image resolution



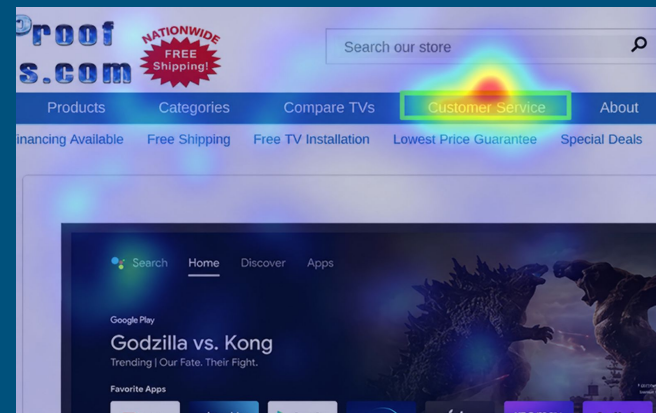
Self-Evolution: Filtering

Given the attention map, two tests:

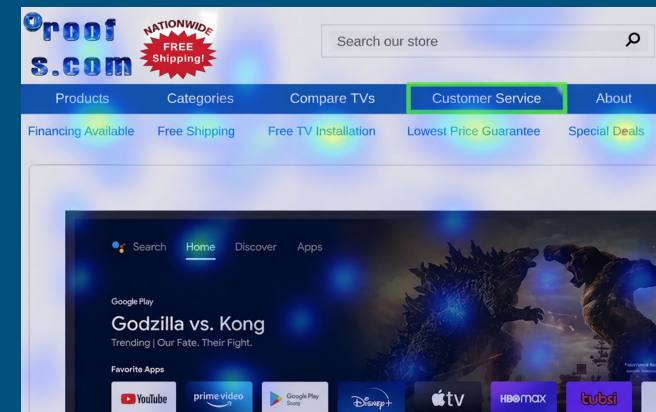
P_{peak} - Is there a significant attention within the bounding box of the element?

P_{global} - Is the average attention within the bounding box greater than the average of the entire image?

Good



Bad



Limitations of SE-GUI

- Aggregation of attention values loses important information
 - Before averaging, each layer is normalized
 - Info on layers with significantly more/less attention to the image tokens in general is lost
- Not all layers need to pay significant attention to the element for P_{peak}
 - Average layers together
 - Majority of layers have to pay significant attention to the element to pass P_{peak}

Limitations of SE-GUI

➤ Noise from output syntax

- All generated output tokens are used to create the attention map
- Most the output is toolcall syntax which does not need to attend to the image

```
<tool_call>

{"name": "computer_use", "arguments": {"action": "left_click",
    "coordinate": [3206, 137]}}

</tool_call>
```

Problem Statement

Want to improve GUI grounding in agentic AI.

Hypothesis:

- I can improve upon SE-GUI's self-evolutionary method with modifications to the filtering method.
- The data filtering for self-evolutionary process can be used for data curation.

Experiment 1 - Self-Evolution Modifications

- Keep relative layer differences in general attention to image tokens
 - No normalization before aggregating
 - Summations instead of averages
- Reasoning on where to click could happen in any layer
 - Pass P_{peak} if **any** layer passes threshold
- Only use attention values corresponding to output coordinates

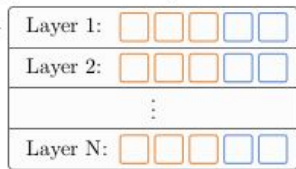
tokenize
sample input



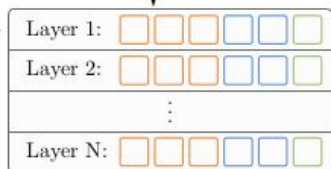
"contact customer service."

Img_{tk1} Img_{tk2} ... Img_{tkM}
 $Instr_{tk1}$... $Instr_{tkL}$

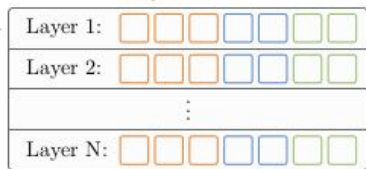
generate output tokens



Out_{tk1}



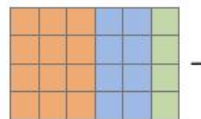
$Out_{tkCoord1}$



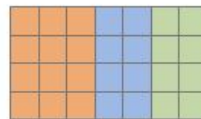
Out_{tkK}

only keep attention
values corresponding
to image tokens

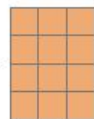
record attention values of
output coordinate tokens
(average heads)



...



sum attention of
output coordinate tokens
(stop and max-norm
for P_{peak})



...



sum layers
and l1-normalize
for P_{global}



project onto
original image
dimensions



Experiment 1 - Method

- used GTA1's published dataset (SE-GUI didn't publish theirs)
- Randomly select 3.2k from GTA1's dataset and train using:
 - SE-GUI training without self-evolution for a baseline
 - Replication of SE-GUI's self-evolution method
 - SE-GUI with our modified self-evolution filtering method
- Evaluate models on ScreenSpot-Pro benchmark

Experiment 2 - Motivation

- Self-evolution filtering starts stages 2, what about applying it to stage 1 for data curation?
- What about combining this data curation and self-evolution?

Experiment 2 - Method

- Use self-evolution filtering method for curation of original 3.2k dataset using base Qwen2.5-VL-3B-Instruct model
- With the curated 3.2k, train using:
 - SE-GUI method without self-evolution
 - SE-GUI with our modified self-evolution filtering method
- Benchmark on ScreenSpot-Pro

Computation Environment

- Used the shared SFSU cluster
- Less resources forced a few key parameter changes

	# A100 GPUs 80G	Epochs per stage	Generations per sample	Samples per step	Batch size
SE-GUI	8	10	8	2	16
Our Experiment	4	4	4	4	16

Results - Experiment 1

Experiment	Stage 1		Stage 2		Stage 3	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
SE-GUI replication	3200	29.79%	265	30.87%	268	31.25%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%

- Unable to replicate SE-GUI's reported results
 - Highly curated dataset and roughly 5x training
- SE-GUI replication did no better than baseline
 - Similar results to baseline with ~1/10 data

Results - Experiment 1

Experiment	Stage 1		Stage 2		Stage 3	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
SE-GUI replication	3200	29.79%	265	30.87%	268	31.25%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%

- Our self-evolve modified filtering outperformed the baseline and replication
 - Modified filtering only removed ~8% of the training samples
 - More robust to lower quality dataset

Results - Experiment 2

Experiment	Epoch 4		Epoch 8		Epoch 12	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
Currated: no self-evolve	2487	32.01%	2487	30.80%	2487	31.50%

Experiment	Epoch 4	
	Train size	Perf.
Baseline: no self-evolve 10k	10000	30.61%
Currated: no self-evolve 10k	7770	33.52%

- Curating works in the short term, but not long term
- Doesn't see the data it initially attended poorly to

Results - Experiment 2

Experiment	Stage 1		Stage 2		Stage 3	
	Train size	Perf.	Train size	Perf.	Train size	Perf.
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%
Currated: no self-evolve	2487	32.01%	2487	30.80%	2487	31.50%
Curation + Self-evolve modified filtering	2487	32.01%	2922	32.32%	2931	32.26%

- Does not perform better than using just the modified self-evolution
 - Open question as to why

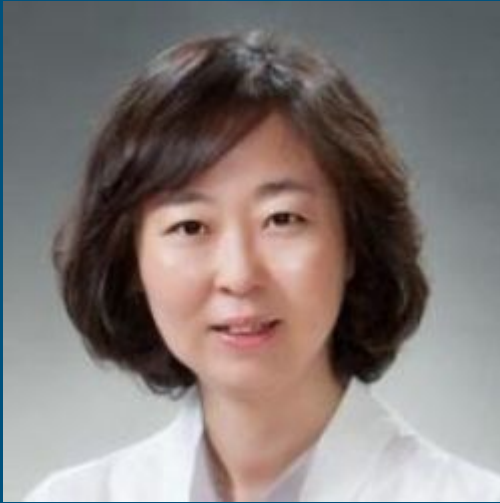
Conclusion

- Our modified self-evolution shows potential
 - Outperformed SE-GUI replication on a lower quality dataset
 - Came close to SE-GUI reported results with 5x less training
- Data quality depends on the capability of the model
 - Data curation only improved performance on the first 4 epochs

Future Work

- Scale it up, we used 3.2K of ~1M desktop samples from GTA1
- Test on SE-GUI curated dataset
- Why did curation with self-evolution modified filtering perform worse than using just self-evolution modified filtering?
 - Does maximizing quantity of data matter more for the earlier epochs and maximizing quality matter more for the later epochs?

Acknowledgements



Dr. Isabel Song
Committee Member



Dr. Akila de Silva
Committee Chair

Acknowledgements

Thanks Jacob Waters for pulling me onto the project leading to this one and your input throughout!



Acknowledgements

SFSU HPC cluster and everyone who shared it during this research.

Acknowledgements



My family in the Bay Area!

Thank You!

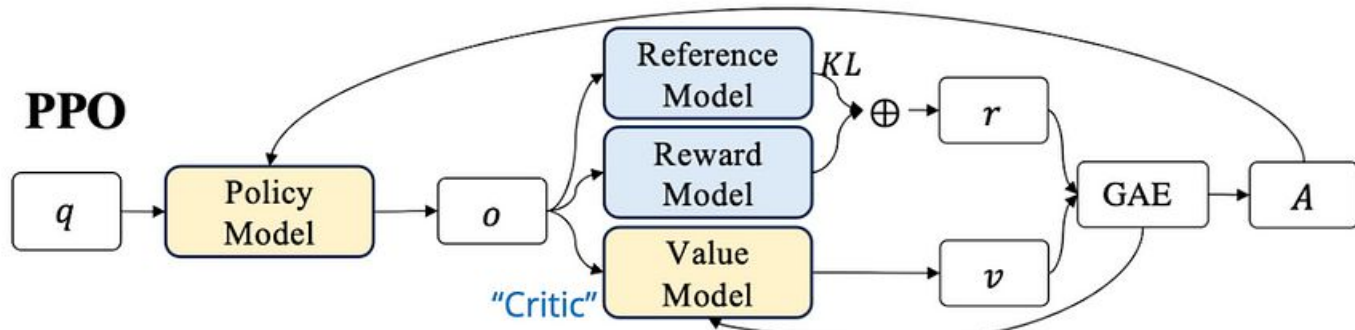
References

- SE-GUI paper: <https://arxiv.org/pdf/2505.12370>
- GTA1 paper: <https://arxiv.org/pdf/2507.05791>
- ScreenSpot-Pro benchmark: <https://arxiv.org/abs/2504.07981>

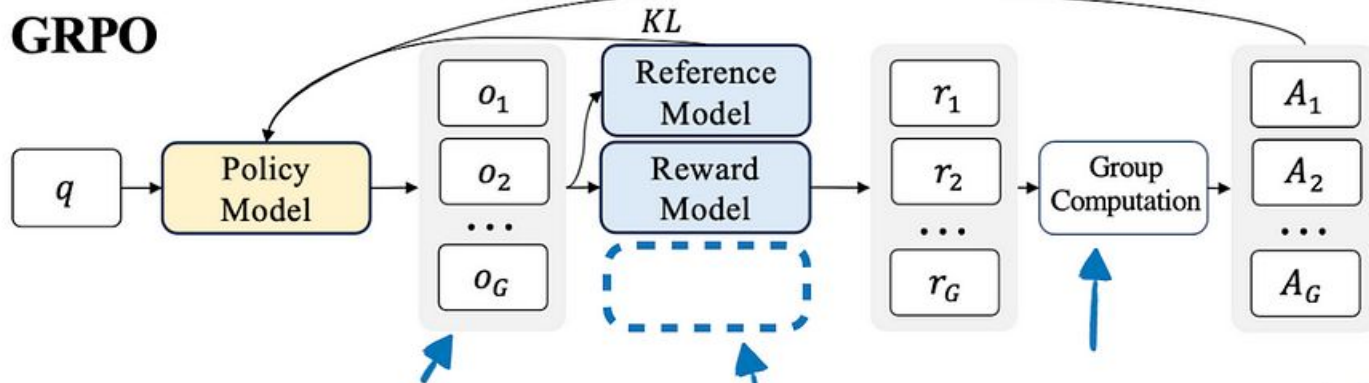
GRPO - Group Relative Policy Optimization

- For each training sample, run through model and generate G predictions
- Using ground truth, calculate their reward and relative advantage to one another
- Update policy based on advantage
- Can update in batches, one update for multiple samples. Helps stabilize training

PPO



GRPO



Sampe multiple answers

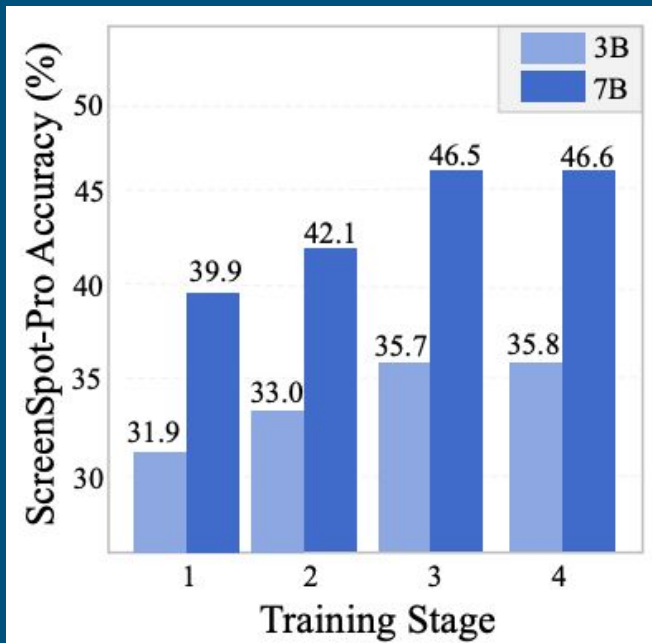
Compute average reward of multiple sampled outputs

GRPO forgoes the critic (value model)

Trained
Models

Frozen
Models

SE-GUI reported results



Results - Experiment 1

Experiment	Stage 1		Stage 2		Stage 3	
	Training set size:	Performance:	Training set size:	Performance:	Training set size:	Performance:
SE-GUI reported results	3,018	31.90%	???	33.00%	???	35.70%
Baseline: no self-evolve	3200	29.79%	3200	31.05%	3200	31.69%
SE-GUI replication	3200	29.79%	265	30.87%	268	31.25%
SE-GUI replication (P_global only)	3200	29.79%	1235	31.06%	1231	31.82%
Self-evolve modified filtering	3200	29.79%	2941	33.71%	2945	34.41%

- P_{peak} is very selective, too selective? What about only filtering with P_{global} ?
- Modified filtering makes it much easier to pass P_{peak} (any layer)